

CDS View 課程練習 15：效能與除錯

Lecture

這一課要教什麼

基礎篇結案時 (cds08) 已經示範過一次「三層 CDS View 比手寫 Open SQL 報表好維護」，但那一課刻意迴避了「效能」這個話題 (只量測傳輸筆數，明確聲明沒做時間 Benchmark)。這一課要正式補上：怎麼量測效能、這系統除錯工具的實際限制、以及這門課從 cds01 到 cds14 一路踩過的真實效能相關發現，整理成一份「常見地雷清單」。

這系統的除錯工具限制 (沿用已查證結論，不重查)

RAP 課程已經查證過：這系統的 **ADT SQL Console** 沒有 **Explain Plan** 功能 (依序找過工具列、選單、右鍵選單都沒有，判斷是這版 ADT 的輕量實作沒有配置這個功能，需要獨立的 SAP HANA Database Explorer)。這一課直接沿用這個結論，不重新查證。可以用的除錯/觀察工具：

- **Eclipse Data Preview**：看查詢結果，[datapreview/freestyle](#) API 回應裡有 [executedQueryString](#)，可以看到框架實際組出的 Open SQL 陳述式 (不是資料庫底層的執行計畫，但至少能確認「框架真的按照你的預期組出查詢」)
- **GET RUN TIME FIELD**：ABAP 語言內建的執行時間量測語句，這一課會示範怎麼用

語法元素講解

GET RUN TIME FIELD：量測一段程式碼的執行時間，單位是微秒：

```
DATA: lv_t1 TYPE i, lv_t2 TYPE i, lv_diff TYPE i.

GET RUN TIME FIELD lv_t1.
" ... 要量測的程式碼 ...
GET RUN TIME FIELD lv_t2.
lv_diff = lv_t2 - lv_t1.
```

⚠ 這系統實測出來的一個小細節：**WRITE** 陳述式的欄位清單裡不能直接寫算術運算式 (例如 **WRITE: / 'elapsed:', lv_t2 - lv_t1.** 會報 **Arithmetic calculation not permitted here**) ——要先把運算結果存進一個變數，才能放進 **WRITE** 清單。

⚠ 這一課量到一個違反直覺的真實結果

用 **GET RUN TIME FIELD** 量測 cds07 的「CDS 聚合」vs.「應用層手動迴圈聚合」，這次量的是**真正的執行時間** (不是 cds07 只量的傳輸筆數)：

CDS aggregation elapsed (microseconds):	23,240	rows:	8
Manual loop elapsed (microseconds):	7,131	raw rows fetched:	356
groups:	8		

CDS 聚合反而比手動迴圈慢了三倍多！這是這一課刻意保留、不迴避的真實量測結果，而不是挑對自己有利的數字。**原因分析：**CDS 聚合這次要多繞一層（先查 `ZI_CDS07_FLIGHT` 明細 View，再疊 `ZC_CDS07_ROUTE_STATS` 聚合 View），每一層都有解析/編譯成本；而手動迴圈版本直接對底層表 `SFLIGHT` 下一句最單純的 `SELECT`，加上這系統的測試資料量只有 356 筆，資料庫執行聚合運算省下的時間，完全被「多一層 View 解析」的固定成本蓋過去。

這一課要傳達的教訓，比「哪個比較快」更重要：「把運算邏輯下推到資料庫」不是無條件的效能萬靈丹，它的效益需要資料量夠大才會顯現（`cds07` 已經示範過，資料量大時傳輸筆數的差距會被放大）；資料量小的時候，額外的層數/框架開銷反而可能讓「看起來更優雅」的做法變得更慢。**效能優化沒有放諸四海皆準的規則，一定要實際量測，不能憑直覺或憑「理論上應該更快」就下結論——**這正是這一課示範 `GET RUN TIME FIELD` 這個工具的意義。

Association 消費與否的效能對照

沿用 `cds03` 建的兩個物件重新量測：

```
Query WITHOUT consuming _Carrier association, elapsed (microseconds):    18,647 |
rows: 5
Query consuming _Carrier association (extra JOIN), elapsed (microseconds): 25,784 |
rows: 5
```

這次的結果符合直覺：多消費一個 Association（觸發額外 JOIN）確實比較慢——跟上一節「CDS 聚合反而較慢」的意外結果放在一起看，更能說明「有些效能直覺是對的（Association 消費 vs. 不消費），有些不是（層數 vs. 手動迴圈），不能一概而論，一定要量」。

這門課從 `cds01` 到 `cds14` 累積的效能相關發現，整理成地雷清單

1. **Association 誤用成不必要的 JOIN**（`cds03` / 這一課）：只要欄位清單裡引用了 Association 底下的欄位，就會觸發 JOIN，即使呼叫端根本用不到那個欄位。設計 Interface View 時，不確定下游會不會用到的關聯資料，優先用「宣告但不消費」的方式（`cds03` 的 `ZI_CDS03_FLIGHT_SCHEDULE` 模式），把「要不要付出 JOIN 成本」的決定權留給消費端。
2. **CDS 疊太多層，不一定是效能問題，但一定要知道自己疊了幾層**（這一課）：層數本身不是萬惡之源（有些層是編譯器強制要求的，見下一點），但每多一層就多一次解析/編譯成本，資料量小的時候這個成本可能比省下來的運算時間還高。
3. **有些「多層」是編譯器強制的，不是設計選擇**（`cds02` / `cds05` / `cds08` / `cds11`）：`CASE WHEN` 不能引用同層計算欄位、聚合函數參數不能是運算式——這兩個限制逼得你一定要多疊一層才能繞過去，這種情境下「減少層數」根本不是一個可行的優化選項，硬要減少反而會撞上編譯錯誤。
4. **聚合下推的效益，要看資料量**（`cds07` / 這一課）：小資料量下，聚合下推可能因為額外層數的固定成本而看起來「沒有比較快」；資料量大的時候，減少傳輸筆數的效益才會真正顯現。

Eclipse ADT 使用建議

1. 對任何 CDS View 用 Data Preview，觀察 `executedQueryString`，確認框架組出的查詢符合預期
2. 懷疑效能問題時，用 `GET RUN TIME FIELD` 實際量測，不要只憑直覺判斷「這樣寫應該比較快」
3. 記得這系統沒有 Explain Plan 工具，資料庫層級的真正執行計畫要靠獨立的 HANA Database Explorer（如果有裝的話），這系統目前沒有

Eclipse ADT Step by Step（重點回顧）

這一課沒有新建 CDS View，重用 cds03 / cds07 已經建好的物件做效能量測示範。如果你想自己動手，可以：

- 1. 對 ZC_CDS07_ROUTE_STATS / ZI_CDS03_FLIGHT_SCHEDULE 等既有物件用 Data Preview，查看 `executedQueryString`
- 2. 寫一支小程序用 `GET RUN TIME FIELD` 量測你自己感興趣的查詢對照組

學習目標

- 能用 `GET RUN TIME FIELD` 正確量測一段程式碼的執行時間，並知道 `WRITE` 陳述式不能直接放算術運算式的限制
- 能講出這系統 ADT SQL Console 沒有 Explain Plan 功能的已知限制，以及替代的觀察手段（Data Preview 的 `executedQueryString`）
- 能舉出這一課量到的違反直覺結果（CDS 聚合在小資料量下比手動迴圈慢），並正確解讀原因（層數的固定成本 vs. 資料量太小無法體現下推效益）
- 能列出這門課從 cds01 到 cds14 累積的至少三個效能相關地雷，並知道哪些「多層」是可以避免的、哪些是編譯器強制的
- 知道效能優化沒有放諸四海皆準的規則，任何直覺判斷都要用實際量測驗證

物件清單

物件	名稱	型別
驗證程式（效能量測）	ZR_CDS15_DEMO	PROG/P

這一課重用 cds03（ZI_CDS03_FLIGHT_SCHEDULE / ZC_CDS03_FLIGHT_WITH_CARRIER）跟 cds07（ZC_CDS07_ROUTE_STATS）已建好的物件，沒有新建 CDS View。ZR_CDS15_DEMO 已建立並啟用。

動手練習（留待後續補做）

- 1. 自己選一組這門課建過的物件（例如 cds08 的三層 View vs. cds08 的 ZR_CDS08_LEGACY_REPORT），用 `GET RUN TIME FIELD` 量一次，看看結果符不符合你的預期
- 2. 對某個 CDS View 用 Eclipse Data Preview，實際找找看有沒有辦法看到 `executedQueryString`（或類似的「框架實際組出的查詢」資訊），回報你在 Eclipse 畫面上實際看到的內容
- 3. 建好後跟我核對

驗證方式

ZR_CDS15_DEMO 透過 `programrun` 無頭驗證，完整量測結果：

```
=== Measurement 1: CDS aggregation (ZC_CDS07_ROUTE_STATS) vs manual ABAP loop aggregation ===
CDS aggregation elapsed (microseconds):      23,240 | rows: 8
Manual loop elapsed (microseconds):           7,131 | raw rows fetched: 356 | groups: 8
(!) Honest caveat: dataset is only 356 rows total - timing differences here are NOT a reliable performance benchmark, they only demonstrate the MEASUREMENT TECHNIQUE (GET RUN TIME FIELD).
=== Measurement 2: Association pitfall - unconsumed vs consumed (recap of cds03
```

```
finding) ===  
Query WITHOUT consuming _Carrier association, elapsed (microseconds):      18,647  |  
rows: 5  
Query consuming _Carrier association (extra JOIN), elapsed (microseconds): 25,784  |  
rows: 5
```

兩組量測都是這系統上真實跑出來的數字，第一組結果違反直覺（如講義說明的原因），第二組結果符合直覺——這一課刻意把兩種結果都保留下來，不刪減對自己「不利」的數字。

動手練習的驗證方式：貼你自己的量測結果給我核對。

思考題

1. 如果把這一課的測試資料量從 356 筆放大到 35,600 筆（100 倍），你預期「CDS 聚合 vs. 手動迴圈」的量測結果會不會反轉？為什麼？
2. 這一課的兩組量測都只跑了一次——單次量測容易受系統當下負載影響而失真，你會怎麼改善這個量測方法讓結果更可靠？
3. 回顧 cds02 / cds05 / cds08 / cds11 學到的「編譯器強制多層」限制，如果你要跟同事解釋「為什麼這個 CDS View 疊了三層」，你會怎麼分辨「這一層是效能考量的設計選擇」還是「這一層是編譯器逼的，沒得選」？

答案

見 [zr_cds15_demo.prog.abap](#)。SAP 端物件：[ZR_CDS15_DEMO](#)（驗證程式，重用 cds03 / cds07 既有物件）。動手練習由你在 Eclipse 動手操作，稍後補做，沒有固定答案快照。